

TypeScript Foundation Sprint — Cheatsheet

A reference of patterns and gotchas learned so far. Organized by concept, not by function — so the lessons transfer to new code.

1. Strict-mode behaviors to keep in mind

- `noUncheckedIndexedAccess` makes `arr[i]` give `T | undefined`. Use `for...of`, `.map()`, destructured `Object.entries()`, or `!` only when the loop is genuinely bounded.
- Named functions over anonymous** — `function foo()` rather than `() =>` for default exports. Stack traces show the name; anonymous shows `<anonymous>`.
- ?? vs || for defaults** — use `??` (nullish coalescing) when you mean "absence of value." `||` falls back on any falsy (`0`, `""`, `false`), which is usually a bug.

```
ts
const factor = Math.pow(10, decimals ?? 0); // ✅ 0 is a valid value
const factor = Math.pow(10, decimals || 0); // ❌ 0 → fallback (lucky here, bug elsewhere)
```

2. Recursion — the two requirements

Every recursive function needs both, or it breaks in confidently-wrong ways:

- A base case that terminates.** Without it, the recursion never ends (stack overflow).
- Each recursive call must move *closer* to the base case.**

The two classic recursion bugs:

Bug	Symptom	Example
Forgot to use the recursive call's return value	Silent wrong answer (data missing/flattened)	<code>deepClone(obj[k])</code> without <code>result[k] =</code>
Forgot to make progress toward the base case	Stack overflow	<code>factorial(n, n*result)</code> instead of <code>factorial(n-1, n*result)</code>

Mental check before running: "Did I use the return value? Does each call get closer to the base case?"

Tail recursion in JavaScript — TS supports the pattern (accumulator parameters), but **V8 (Node/Chrome) does NOT implement TCO**. So tail recursion in JS is *stylistic*, not *stack-safe*. For unbounded n, use a loop:

```
ts

function fibonacci(n: number): number {
  let [a, b] = [0, 1];
  for (let i = 0; i < n; i++) [a, b] = [b, a + b];
  return a;
}
```

3. Type guards (predicates)

The shape: `x is T` in the return type. Returns boolean; narrows the type inside `if (...)`.

```
ts

function isString(x: unknown): x is string {
  return typeof x === 'string';
}
```

The unsoundness trap with generics: TS types are erased at runtime, so generic predicates like `<T>(x): x is T[]` can lie — at runtime you can only check "is it an array?", not "is it an array of T". Two honest fixes:

```
ts

// Option A - be honest about what runtime can prove
function isArrayOfUnknown(x: unknown): x is unknown[] {
  return Array.isArray(x);
}

// Option B - require a per-item guard, actually verify items
function isArrayOf<T>(x: unknown, isItem: (v: unknown) => v is T): x is T[] {
  return Array.isArray(x) && x.every(isItem);
}

// Composes: isArrayOf(x, isNumber)
```

Adding to a type vs replacing it: when your predicate adds info to an object, intersect (`T & ...`) instead of replacing — preserves existing fields, especially in generic contexts:

```
ts
```

```
// ❌ replaces – loses other fields in generic contexts
function hasKey<T extends string>(obj: object, key: T): obj is Record<T, unknown>

// ✅ intersects – keeps original info
function hasKey<O extends object, K extends string>(
  obj: O, key: K
): obj is O & Record<K, unknown>
```

4. Assertion functions (asserts x is T)

Shape: `asserts x is T` in the return type. The function returns void (or throws). After the call, TS narrows for the **rest of the scope** — no `if` needed.

```
ts

function assertDefined<T>(x: T | null | undefined): asserts x is T {
  if (x === null || x === undefined) throw new Error("Expected value");
}

// Usage:
assertDefined(maybeUser);
maybeUser.name; // ✅ narrowed for the rest of the scope
```

	Type guard <code>x is T</code>	Assertion function <code>asserts x is T</code>
Returns	<code>boolean</code>	<code>void</code> (or throws)
Narrowing scope	inside <code>if (...)</code> block	rest of the scope after the call
Use when	bad case is expected, want to handle	bad case is a bug, crash loudly

Classic uses for `assertDefined`: `document.getElementById` (returns `HTMLElement | null`), `Array.find` (returns `T | undefined`), config values you know are set.

5. Narrowing toolkit (in order of specificity)

Tool	What it narrows	Notes
<code>x === null x === undefined</code>	Nullish	The <code>==</code> shortcut <code>x == null</code> also works but is hated by some linters

Tool	What it narrows	Notes
<code>typeof x === 'string'</code>	Primitive types	<code>'string'</code> , <code>'number'</code> , <code>'boolean'</code> , <code>'undefined'</code> , <code>'object'</code> , <code>'function'</code> , <code>'symbol'</code> , <code>'bigint'</code>
<code>Array.isArray(x)</code>	Arrays	More reliable than <code>instanceof Array</code> (cross-realm safe)
<code>x instanceof Map</code>	Class instances	<code>Map</code> , <code>Set</code> , <code>Date</code> , custom classes
<code>'foo' in x</code>	Property existence	Walks prototype chain
Custom type predicate <code>(x is T)</code>	Anything	When none of the above fit

Branch order matters — narrow from most-specific to least-specific. Arrays before plain objects (every array is `typeof === 'object'`), Maps before objects, etc.

6. Discriminated unions + `assertNever`

The exhaustiveness pattern — compiler-enforced "did I handle every case?":

```
ts

function assertNever(x: never): never {
  throw new Error(`Unhandled case: ${JSON.stringify(x)}`);
}

type Shape =
  | { kind: "circle"; radius: number }
  | { kind: "square"; side: number };

function area(shape: Shape): number {
  switch (shape.kind) {
    case "circle": return Math.PI * shape.radius ** 2;
    case "square": return shape.side ** 2;
    default: return assertNever(shape); // ← compile-time exhaustiveness
  }
}
```

If someone adds a new variant to `Shape` and forgets to handle it, TS will refuse to compile — `shape` in the default would be that variant, not `never`, and `assertNever` requires `never`.

Where you'll see it: state machines / reducers (Redux/Zustand), async fetch state (idle/loading/success/error), notification systems, WebSocket message handlers — anywhere you have a `kind`/`type`/`status` field driving a switch.

7. Generic constraints — the `extends` family

```
ts
T extends object           // any non-primitive — usually what you want
T extends keyof object     // 🚫 TRAP — evaluates to `never`, function can't be called
T extends readonly unknown[] // T is an array type itself
```

`T` vs `T[]` — when the constraint already says `T` is an array (`T extends readonly unknown[]`), use `opts: T`, not `opts: T[]`:

```
ts
// ❌ array of arrays
function isOneOf<T extends readonly unknown[]>(v: unknown, opts: T[]): v is T[number]

// ✅ T is the array, T[number] is the element
function isOneOf<T extends readonly unknown[]>(v: unknown, opts: T): v is T[number]
```

The `as const` payoff for literal-union narrowing:

```
ts
const STATUSES = ["pending", "approved", "rejected"] as const;
type Status = typeof STATUSES[number]; // "pending" | "approved" | "rejected"

function setStatus(s: string) {
  if (isOneOf(s, STATUSES)) {
    // s is now Status — a narrow literal union, not just string
  }
}
```

8. Object operations — own vs inherited

JavaScript objects have a **prototype chain**. Inherited properties are accessible but don't *belong* to the object.

Tool	Question it answers
<code>obj.prop</code>	"Can you find this property anywhere up the chain?" — walks chain
<code>'prop' in obj</code>	"Is this property accessible?" — walks chain
<code>Object.keys(obj)</code>	"What properties does obj OWN?" — own only
<code>Object.hasOwn(obj, key)</code>	"Does obj own this key?" — own only, ES2022
<code>for (key in obj)</code>	"What can I enumerate?" — walks chain

`Object.hasOwn` over `obj.hasOwnProperty` — the latter is shadowable (`{hasOwnProperty: 'gotcha'}.hasOwnProperty('foo')` crashes). `Object.hasOwn(obj, key)` is the safe modern equivalent.

`Object.keys` returns `string[]` in TS even for `{a: 1}` — for sound reasons (TS is structural, extra keys may exist at runtime). Cast at the boundary if you need typed keys:

```
ts
function typedKeys<T extends object>(obj: T): (keyof T)[] {
  return Object.keys(obj) as (keyof T)[];
}
```

The same one-line-with-cast shape works for `typedValues`, `typedEntries`.

9. Floating point & number precision

JavaScript numbers are IEEE 754 floats. Two things to know:

1. Decimals can't always be exactly represented. `0.1 + 0.2 === 0.30000000000000004`. To round safely, nudge with `Number.EPSILON`:

```
ts
function roundTo(num: number, decimals: number): number {
  const factor = Math.pow(10, decimals ?? 0);
  return Math.round((num + Number.EPSILON) * factor) / factor;
}
```

2. Integer precision dies past `253 - 1` (~9 quadrillion). Beyond that, use `BigInt`:

```
ts
```

```
Number.MAX_SAFE_INTEGER === 9_007_199_254_740_991;
// fib(78) is the last fully-accurate Fibonacci in `number` type
```

Financial code: use integers-as-cents or `decimal.js`. Never `number`.

10. Spread, shallow vs deep

`{...a, ...b}` and `[...arr]` are **shallow**. Nested objects/arrays share references with the source.

```
ts
const a = { inner: { count: 1 } };
const merged = { ...a };
merged.inner === a.inner;           // true - same reference
merged.inner.count = 999;
a.inner.count;                      // also 999 🤖
```

Deep clone: use `structuredClone(obj)` in Node 17+ / all modern browsers. Handles cycles, Maps, Sets, Dates correctly.

Deep merge: no built-in — write recursion or use `_.merge`.

11. Testing patterns

Pass the function, don't call it, when testing throws/async:

```
ts
expect(() => fn()).toThrow();           // ✅
expect(fn).toThrow();                   // ✅ direct reference
expect(fn()).toThrow();                 // ❌ throws before expect sees it
expect(async () => await fn()).rejects.toThrow(); // async
```

Tests describe a contract, not just code. When you intentionally change behavior, update the tests to match the new contract. A failing test isn't always "code wrong" — sometimes it's "test was checking the old promise."

Test the falsy-but-defined values (`0`, `false`, `""`) when checking "is it defined?" — they're the classic trap for naive `if (!x)` implementations.

Test the use case, not just the unit. For `assertNever`, `assertDefined`, type guards — write tests that exercise the function inside its real pattern (switch/find/null-check), not just in isolation.

Group tests by contract with `describe` blocks:

```
ts

describe("isEmpty", () => {
  describe("nullish", () => { ... });
  describe("strings", () => { ... });
  describe("non-integers - by contract, always false", () => { ... });
});
```

The structure itself documents what the function promises.

12. General engineering heuristics

- "What cases am I NOT testing?" — passing tests $\neq$ correct code. Always probe: empty input, single-element, mutation, boundary values, the surprising-but-legal value (NaN, Infinity, negative, zero).
 - **Iterate the smaller / more specific collection.** For `pick(obj, keys)`, iterate `keys`, not `obj`. For `mapValues`, the input keys are already unique so collision is impossible.
 - **When you find yourself reimplementing a built-in** (`Object.keys`, `Object.entries`, `Object.values`) — the value-add is the *types*, not the *logic*. One-liner with a cast is usually the right answer.
 - **Contract questions on every function:** what about `null` / `undefined`? Empty input? Negative numbers? Non-integers? Decide loudly and write a test for the decision.
-

13. Quick patterns I want to remember

```
ts
```

```
// Typed wrappers around Object built-ins
function typedKeys<T extends object>(obj: T): (keyof T)[]
function typedValues<T extends object>(obj: T): T[keyof T][]
function typedEntries<T extends object>(obj: T): [keyof T, T[keyof T]][]

// The two narrowing shapes
function isXxx(x: unknown): x is Xxx           // type guard
function assertXxx(x: unknown): asserts x is Xxx // assertion

// Exhaustiveness boilerplate
function assertNever(x: never): never {
    throw new Error(`Unhandled: ${JSON.stringify(x)}`);
}

// Literal-union list (idiomatic)
const COLORS = ["red", "green", "blue"] as const;
type Color = typeof COLORS[number];

// Adding to an object type without replacing
obj is 0 & Record<K, unknown>    // ✅ adds
obj is Record<K, unknown>       // ❌ replaces (loses generic info)
```